

OBJEKTORIENTERET ANALYSE, DESIGN OG IMPLEMENTATION

5: Abstractions, interfaces and polymorphism

Jan H. Mikkelsen

Cyber og Computerteknologi

2. semester – KBH – F26

AGENDA

1

- A wee recap of assignment(s) from last time?
- SOLID principles behind OO design
- Abstract classes
- Interfaces
- Polymorphism
- The important stuff
- Assignments

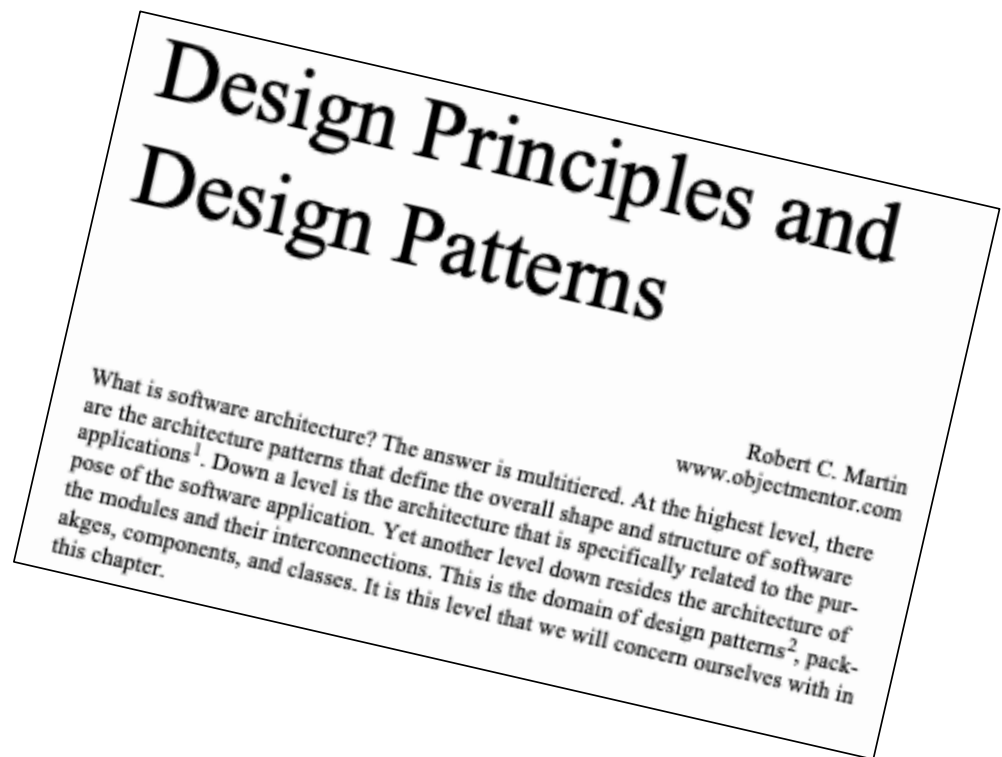


- To understand SOLID
- To understand and be able to use abstract classes
- To understand and be able to use interfaces
- To understand and be able to use polymorphism

SOLID – 5 ESSENTIAL PRINCIPLES OF OO

2

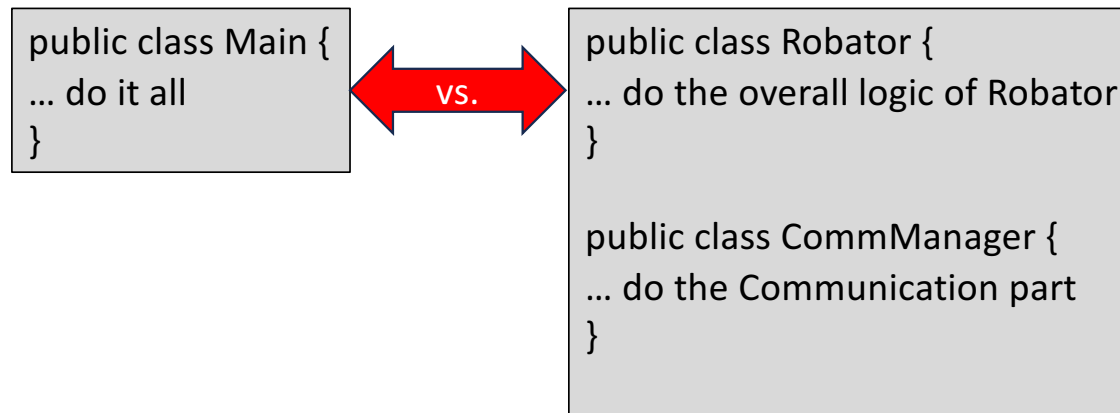
- In addition to the objectives, such as platform independence etc., of OO, as we already discussed – a set of five principles have also been defined
- SOLID
 - S - Single-responsibility principle
 - O - Open-closed principle
 - L - Liskov Substitution principle
 - I - Interface Segregation principle
 - D - Dependency Inversion principle



SOLID – SINGLE-RESPONSIBILITY PRINCIPLE

3

- A class should have one and only one reason to change, meaning that a class should have only one job
- This approach will automatically lead to a simple code structure, as your code will be broken into smaller (smallest?) parts
- Be aware that methods should also naturally belong to the class itself



SOLID – OPEN-CLOSED PRINCIPLE

4

- Objects or entities should be open for extension but closed for modification
- We may always need to extend
- But changes to current, operational code can be a nightmare
 - Particularly if we want to change method names
- If we do any implementation changes, we should keep it inside our method
 - Use interfaces
- Here, whenever we need to add communication protocols we must modify the class

```
public class CommManager {  
    public void wifiConnect() { /* ... */ }  
}  
// or  
public class CommManager {  
    public void bluetoothConnect() { /* ... */ }  
}
```

SOLID – OPEN-CLOSED PRINCIPLE

5

- Instead, we should form an interface as part of the main class and then extend/overwrite this in the actual protocol-specific classes
- Using 'final' we prevent further sub-classes
- If a super-WiFi is to be added, then this MUST come from an additional class definition

```
public interface CommManagerIF {  
    void connect();  
}  
  
public final class WifiComm implements CommManagerIF {  
    @Override public void connect() { // Wi-Fi specific connection  
    }  
}  
  
public final class BluetoothComm implements CommManagerIF {  
    @Override public void connect() { // Bluetooth specific connection  
    }  
}
```

SOLID – LISKOV SUBSTITUTION PRINCIPLE

- Let $q(x)$ be a property provable about objects of x of type T . Then $q(y)$ should be provable for objects y of type S where S is a subtype of T
- If something is true for *every* object of type T , then it must also be true for *every* object of type S , where **S is a subclass of T**
- We can treat both Circle and Square as Shapes
- A critical aspect of inheritance
- Where things break
 - Bird has a method `fly()` ←

move() would be okay here
 - You create Penguin : Bird
 - Penguin.fly() throws an exception
- LSP ensures (more) predictable and maintainable code

```
public class Shape{  
    public void draw();  
}
```

```
public class Circle extends Shape{  
    public void draw();  
}
```

```
public class Square extends Shape{  
    public void draw();  
}
```

SOLID – INTERFACE SEGREGATION PRINCIPLE

7

- A client should never be forced to implement an interface that it does not use, or clients should not be forced to depend on methods they do not use
- Should be clear – if a method is not needed for a subclass, it should also not be forced to implement it
- In this example, it makes no sense that one can call the method `getRadius()` for the Square class
 - What is the radius of a square?
- The solution (or one solution at least) is to segregate (split) the interfaces based on the qualities/characteristics the shape actually has

```
public class Shape{  
    public void draw();  
    public double getRadius();  
}
```

```
public class Circle extends Shape{  
    public void draw();  
}
```

```
public class Square extends Shape{  
    public void draw();  
}
```

SOLID – INTERFACE SEGREGATION PRINCIPLE

8

- It is advisable to define the split interfaces outside Shape but then within the same package

- shapes/
 - Shape.java
 - Circular.java
 - Circle.java
 - Square.java

```
package shapes;

public interface Shape {
    double getArea();
}
```

```
package shapes;

public interface Circular {
    double getRadius();
}
```

```
public class Circle implements Shape, Circular {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double getRadius() {
        return radius;
    }

    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }
}
```

SOLID – INTERFACE SEGREGATION PRINCIPLE

9

- Keyword extends is used for class inheritance
- What it means:
 - You inherit fields and methods from the parent class
 - You can override methods
 - You can reuse existing behaviour
 - You have an is-a relationship
 - A 'Circle' is-a 'Shape'
 - You can only extend ONE class (Java restriction). Two classes can have different implementation of a method for instance .. which one is (poor) Java to use then?
 - You may implement as many interfaces as you want though!

```
public class Shape {  
    public double getArea() { return 0; }  
}  
  
public class Circle extends Shape {  
    @Override  
    public double getArea() {  
        return Math.PI * radius * radius;  
    }  
}
```

SOLID – INTERFACE SEGREGATION PRINCIPLE

10

- Keyword `implements` is used to ensure contract compliance
- What it means:
 - Here you **promise** to implement the methods in the interface
 - No behaviour is inherited - only the method *signatures*
 - You can implement **multiple** interfaces
 - Indicates a **can-do** capability rather than inheritance

```
public interface Shape {  
    double getArea();  
}  
  
public interface Circular {  
    double getRadius();  
}  
  
public class Circle implements Shape, Circular {  
    // must implement both getArea(  
    // and getRadius()  
}
```

SOLID – DEPENDENCY INVERSION PRINCIPLE

11

- Entities must depend on abstractions, not on concretions. It states that the high-level module must not depend on the low-level module, but they should depend on abstractions
- The point here is to ensure decoupling between classes
- If draw() is implemented in the parent class, with a circle in mind, then the square will be drawn as a circle
- Generalization of interfaces goes upwards and specialization goes downward in a class hierarchy
- This is a generic abstraction in system design, but it is especially true for code

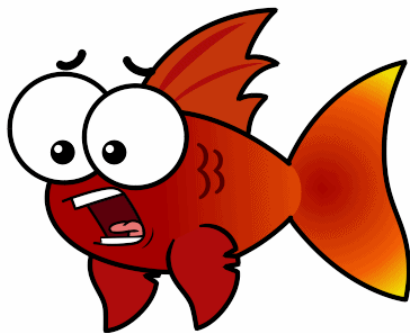
```
public class Shape {  
    public void draw();  
}
```

```
public class Circle extends Shape {  
}
```

```
public class Square extends Shape {  
}
```

BREAK – PAUSE(FISK)

12



Jan H. Mikkelsen, OOAD15, CoCT, 2026

AGENDA

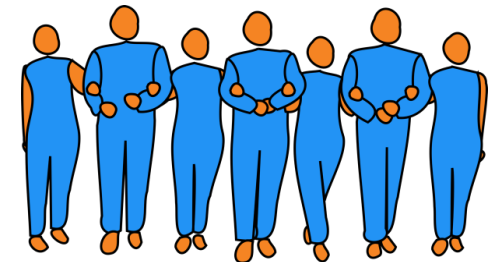
13

- A wee recap of assignment(s) from last time?
- SOLID principles behind OO design
- Abstract classes
- Interfaces
- Polymorphism
- The important stuff
- Assignments

CONSIDER THE FOLLOWING CASE

14

- We develop in teams and are tasked to implement a drawing program
 - Person A gets to implement the code that calculates squares and circles
 - Person B gets to implement the code that calculates polygons
 - Person C gets to implement the visualization part
- Person A adds methods `drawSquare()` and `drawCircle()` as this makes sense to him
- Person B adds `drawPolygons()` to her class
- Person C is confused and must implement a lot of if-statements depending on which shape is drawn
- Team manager decides that all must use a method `draw()`, and the implementation responsibility is given to Person C



ABSTRACT CLASSES

15

- Abstract classes are classes we cannot directly use
 - Wait? – What? Why? Is this even useful?
- In certain situation, such an abstraction actual makes sense
- Consider the case where we:
 - need to make code for drawing Circles and Squares and use it in our code
 - can be sure that shape is properly implemented if we extend Circle from Shape as well with Square if we extend Shape
 - can identify both Circle and Square as Shapes via the keyword ***instanceof***, and do stuff like
 - If (s1 instanceof Shape){..s1.draw()..}

```
abstract class Shape() {  
    abstract void draw();  
    setCenterCoordinate(x,y);  
}
```

```
class Circle extends Shape {  
    void draw() { ... }  
    getRadius();  
}
```

.... code

```
Shape s1= new Circle();  
s1.setCenterCoordinate(0,0);  
s1.draw();  
Shape s2= new Square();  
s2.draw();  
s2.setCenterCoordinate(3,3);
```

ABSTRACT CLASSES

- Abstract classes are *not* about overriding — they are about enforcing a contract + sharing partial implementation
- Abstract classes let you define a contract *and* shared logic
- Abstract classes are handy when
 - some behaviours must be provided by subclasses
 - some behaviour is common and should be inherited
 - some internal state(s) must be shared
 - some methods should be left for implementation in sub-classes

```
abstract class Animal {  
    // Force subclasses to implement  
    abstract void makeSound();  
  
    // Shared implementation  
    void sleep() {  
        System.out.println("Sleeping...");  
    }  
}
```

ABSTRACT CLASSES

- Using abstract classes, we prohibit any instances of that class from being formed
 - E.g., we cannot have an object Shape, but we need to have a Circle or a Square. However, we can still check if either one is a Shape
 - We can add as many other common things as normal, which typically is what we want to do anyway
 - E.g., Shape can have a method called setCoordinate(x0,y0) to set its center coordinate to somewhere
 - draw() needs specific implementations as squares and circles are differently – but in both cases it makes sense to draw the shape
- The main goal of an abstract class is to provide shared implementation of common behaviour - promoting the reuse of code

ABSTRACT CLASSES

18

- An abstract class defines a **common base** for related subclasses
 - It outlines shared behaviour or expectations across that family
 - An abstract class therefore relates to a family of inheritance
- We may see it as an incomplete inheritance, where you force your children to implement specific code with a common interface
 - It implements **some** behaviour and leaves **other** behaviours abstract (unimplemented) – an abstract class is therefore indeed “incomplete”
 - Subclasses must provide the missing pieces – ensuring the shared contract
- Interestingly though, one can also have abstract classes with static methods, which basically bypass all OO principles

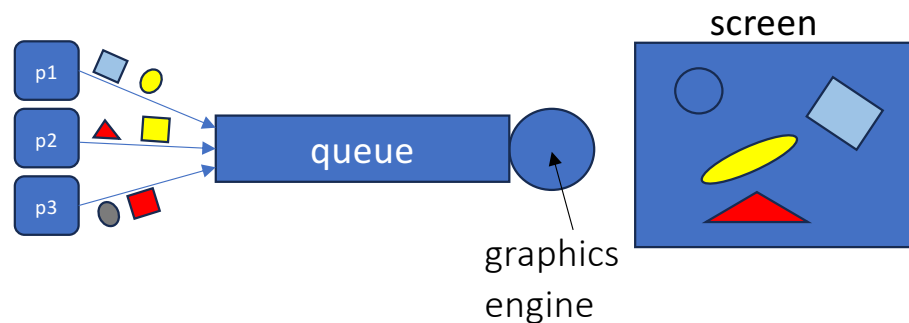
```
abstract class MathHelper {  
    static double circleArea(double r) { return Math.PI * r * r; }  
}
```

```
MathHelper.circleArea(5)
```

Procedural programming
in disguise!!!!

ABSTRACT CLASSES

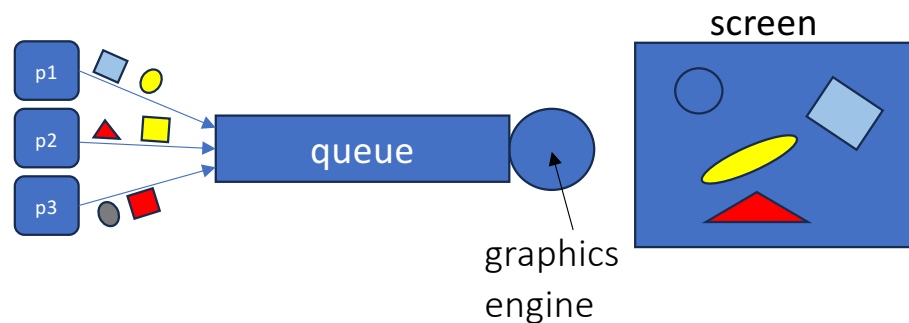
- A practical example of such bypassing behaviour is seen in the producer/consumer scenario in GUI systems
 - Producers create work
 - A graphics engine processes work in order
 - The screen displays the result
- In OOD, each shape would normally be represented by **objects** with behaviours — typically subclasses of a base class or implementations of an interface (e.g., Shape)



```
abstract class Helper {  
    static void demofun() {  
        System.out.println("Geeks for Geeks");  
    }  
}  
  
public class GFG extends Helper {  
    public static void main(String[] args) {  
        Helper.demofun();  
    }  
}
```

ABSTRACT CLASSES

- However, in this example where a static method is placed inside an abstract class, the whole object-oriented design is bypassed
 - Helper is abstract and hosts a static Helper.demofun()
 - GFG *extends* Helper but it does not use inheritance in any meaningful OO way - it just calls the static method via Helper.demofun()
- Helper.demofun() is essentially just a global function that happens to live inside a (random) class
- Abstract classes are for incomplete OO inheritance



```
abstract class Helper {  
    static void demofun() {  
        System.out.println("Geeks for Geeks");  
    }  
}  
  
public class GFG extends Helper {  
    public static void main(String[] args) {  
        Helper.demofun();  
    }  
}
```

ABSTRACT CLASSES

21

- OOP suggest:
 - Each Shape object draws itself
 - The screen is built by asking each object to render
 - Rendering is just lots of `shape.draw(graphicsContext)` calls
 - “Each object knows how to do its own job”
- REAL GUI rendering systems:
 - Collect draw commands, batch, sort them and flattens hierarchy into a linear work queue
 - Feed the queue to a renderer
 - Send optimized commands to the GPU
 - This is *not* object-centric but rather *pipeline-centric*

for each Shape:
 shape.draw()

for each shape:
 extract data from shape
 sort/batch data
 send to GPU-compositor

ABSTRACT CLASSES

- Abstract classes are *tools of classical OOP*
 - Abstract classes represent *the purest, textbook* OOP model
 - You create a base class with behaviour
 - Subclasses override or extend the behaviour
 - Each object is responsible for its own operations
 - Polymorphism dispatches to the right subclass
 - State + behaviour are in the same place
 - “Call `shape.draw()` and the right thing happens”
 - In other words, Abstract classes = the OOP idea that “objects do the work.”
- OOP is not *fast enough* for the core performance-critical parts, but it *is still extremely useful* for architecture, APIs, and high-level organization
 - High-performance engines *do* use OOP — just not where speed matters

AGENDA

23

- A wee recap of assignment(s) from last time?
- SOLID principles behind OO design
- Abstract classes
- Interfaces
- Polymorphism
- The important stuff
- Assignments

INTERFACES

- Interfaces are specification of methods and they tell us
 - what methods exist
 - what their signatures look like
 - what behaviour a class must provide

```
interface Animal {  
    public void animalSound(); // interface method (does not have a body)  
    public void run(); // interface method (does not have a body)  
}
```

- They do not allow code (like function definitions in C-header files)
- Interfaces separate *what* something does from *how* it does it and allows code to depend on **abstractions**, not concrete classes

INTERFACES

25

- And why is this separation useful?

```
//file: shapes/Shape.java
package shapes;

public interface Shape {
    void draw(java.awt.Graphics2D g);
    // or return an area, etc.
}
```

- draw() does not depend on the specific shape (D in SOLID)
- The interface can be small and precise
- No object is forced to implement interfaces it does not need (I in SOLID)

```
//file: shapes/Square.java
package shapes;

public class Square implements Shape {
    private final double side;
    public Square(double side) { this.side = side; }
    @Override
    public void draw(java.awt.Graphics2D g) {
        // drawing logic...
    }
}
```

```
//file: shapes/Circle.java
package shapes;

public class Circle implements Shape {
    private final double radius;
    public Circle(double radius) { this.radius = radius; }
    @Override
    public void draw(java.awt.Graphics2D g) {
        // drawing logic...
    }
}
```

INTERFACES

- Further exemplification using the `getRadius()` case from earlier
- Here, the `Square` class does not implement `getRadius()`

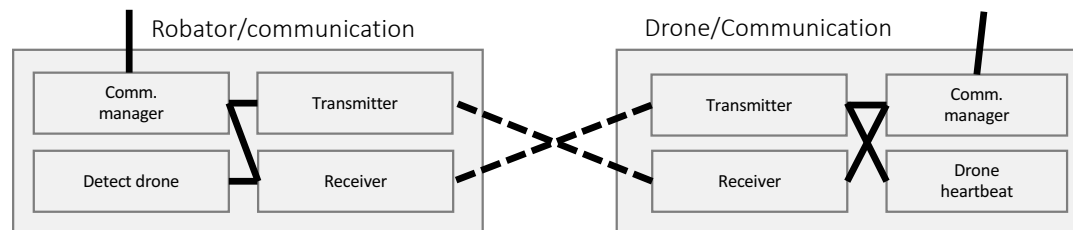
```
public interface Shape {  
    void draw(Graphics2D g);  
    double area();  
}  
  
public interface Circular {  
    double getRadius();  
}
```

```
public class Circle implements Shape, Circular {  
    private double radius;  
    public Circle(double r) { this.radius = r; }  
  
    @Override  
    public void draw(Graphics2D g) { ... }  
    @Override  
    public double area() { return Math.PI * radius * radius; }  
    @Override  
    public double getRadius() { return radius; }  
}  
  
public class Square implements Shape {  
    private double side;  
    public Square(double side) { this.side = side; }  
  
    @Override  
    public void draw(Graphics2D g) { ... }  
    @Override  
    public double area() { return side * side; }  
}
```

DESIGNING USING INTERFACES

27

- Assuming again that we are a team of developers spread geographically all across the world
- We agree to implement each part of the communication manager



Denmark



Canada



Bangladesh

- What happens if we meet after 3 months?

DESIGNING USING INTERFACES

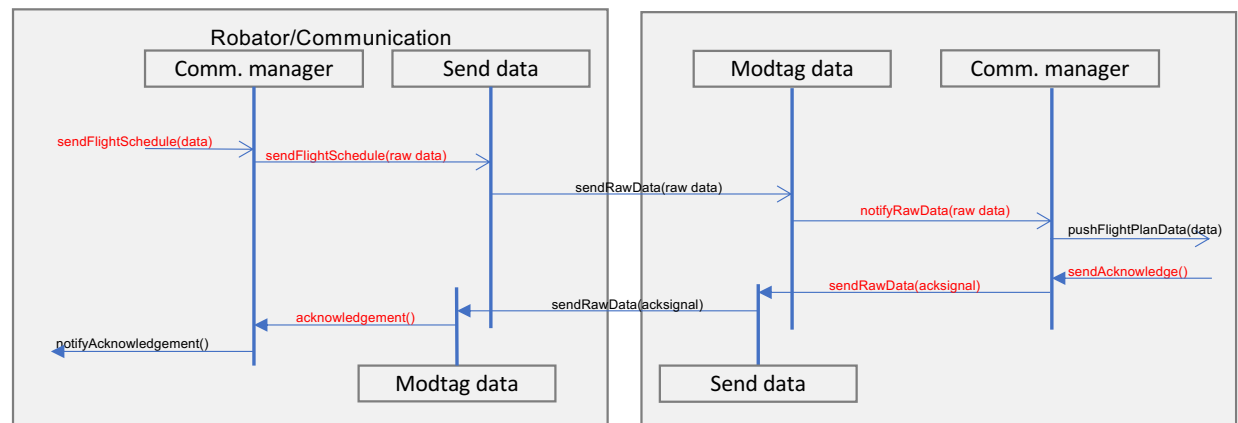
28

- It will likely be a "Huston – we have a problem" scenario, as for the Apollo 13 – the unlucky mission where everything went wrong
- *We have to make this fit into this, using only that on the table!!!*
- Go....
 - Oh, and we must solve this before they die!!!



DESIGNING USING INTERFACES

- How to avoid problems like that?
- Well .. BEFORE starting coding, we must agree on interfaces
- Luckily, we have done our homework with proper design and have already created message sequence diagrams
- This provides a firm basis for defining what interfaces our communication manager must have



DESIGNING USING INTERFACES

30

- The strength lies in the agreement!
- If we can just agree on the interfaces, based on our message sequence diagrams
 - This can be surprisingly hard, as we actually need to discuss with each other
 - A good initial design on paper with a pen helps a lot!!!
- Once we agree, we can count on these methods being implemented
 - How it is done, the actual implementation, is less important, this is up to the individual teams to worry about

Team Denmark

```
interface CommManager {  
    public int sendFlightSchedule(Data data);  
    public void acknowledgement();  
}
```

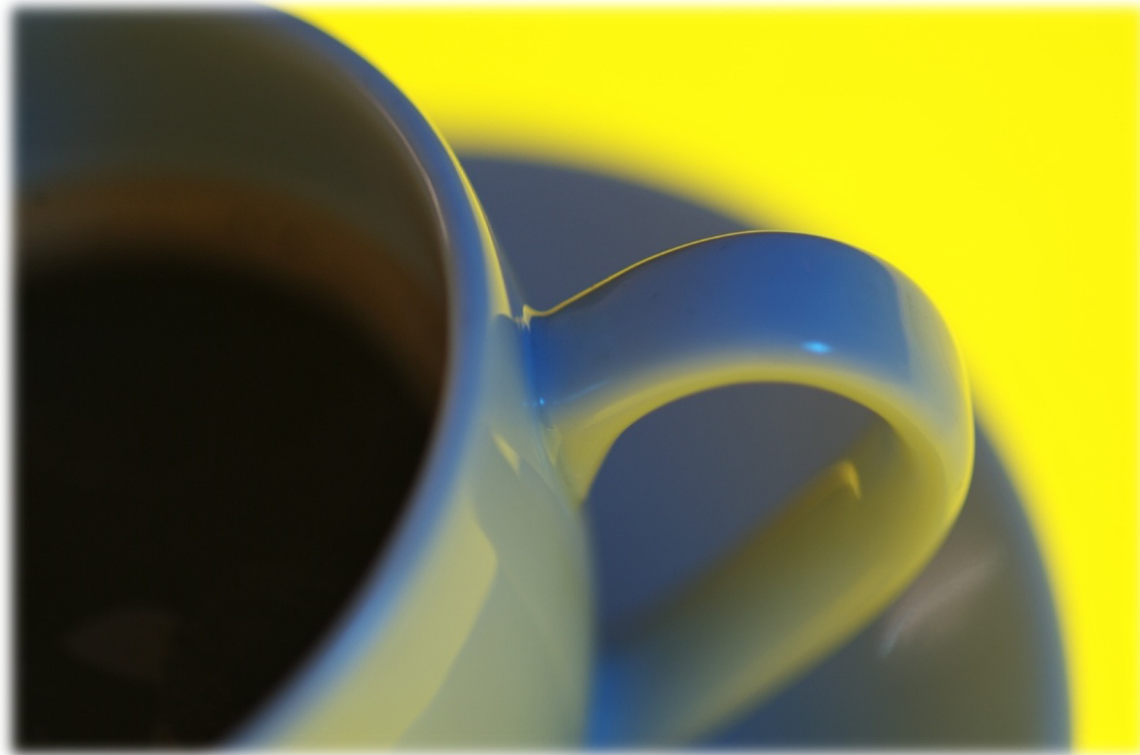
Team Canada

```
interface SendData {  
    public int sendFlightSchedule(Byte[] data);  
}
```

Team Bangladesh

```
interface ReceiveData {  
    public int setTimeout(float timeout);  
}
```

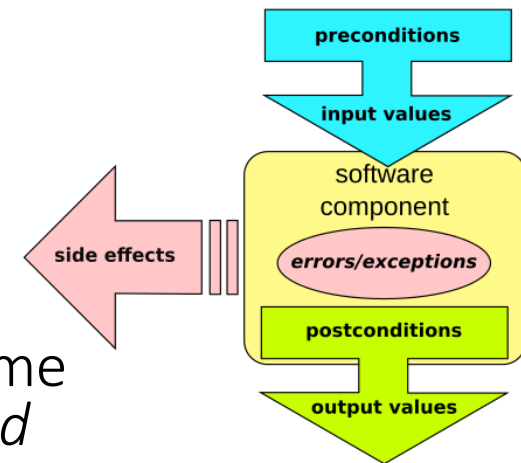
"BIKER BREAK!"



WHERE IS THE OO IN THE INTERFACE PART

32

- Is this not just modularization of software components?
- Both the open-closed principle and Liskov substitution principle supports the use of interfaces
- Design-by-contract is a concept introduced in 1986, credited to Bertand Meyer
 - Like module specification/requirement
 - Input, output, functionality
- The point being – we are all trying to solve the same problem and *we want to deliver what we promised*



JAVA INTERFACES

33

- What is a Java interface?
 - A more restrictive version of the general OO definition
 - Strong separation between state and behaviour
- In short, it is just a list of methods and attributes without any code
 - Similar to header files in C and C++ where we define functions (and no code)
- A class can then implement an interface
 - Class = behaviour + state

```
public interface Vehicle {}
```

```
public interface Vehicle {  
    public void driveForward();  
    public void brake();  
    ....  
}
```

```
public class Bicycle implements Vehicle{}
```

JAVA INTERFACES

34

- While DriveForward and Brake are similar methods for vehicles in general, they are very different in the way they are implemented for a bike or a car
- DriveForward:
 - Car: pressing a speeder makes the engine work and drives the wheels
 - Bike: bicyclist steps on the pedals, which drives the wheels
- I can be unsure if I am handling a Car or a Bike, but it is extremely useful to know that I am handling a Vehicle
 - If I have a list of classes that all implements Vehicle, then I can for ALL of them be sure that a call to DriveForward would work
 - I can even cast to the interface

```
Vehicle vehicleIF = (Vehicle) objectInList[i];  
vehicleIF.driveForward();
```

List of vehicles in
array objectInList[]

Volvo
Racercycle
Train
Tesla
Scooter

INTERFACE VS. INHERITANCE

Interface

- Allows individual implementations of same method
- Forces all classes to have specific and same methods
- Classes can implement more than one interface
- Attributes can only be defined as *final static*
- Interfaces are only fully supported if made *public* (private and protected have some restrictions, and are, in earlier java, completely prohibited)

Extends (inheritance)

- Allows to reuse code for subclasses, so you only need to write code once
- Classes can only inherit from one class, but a class can have multiple subclasses
- Attributes can easily be inherited by subclasses and access controlled via keywords *public*, *protected*, *private*

CAN I USE ONLY ONE OR ANOTHER

- Both inheritance and interfaces can be used at the same time
 - We can still only inherit from one parent class at a time
 - But there's virtually no limit to how many interfaces we can implement
- Simply start with listing your parent class first, then your interfaces, comma-separated, like so:
 - Class Example : ParentClass, InterfaceOne, InterfaceTwo

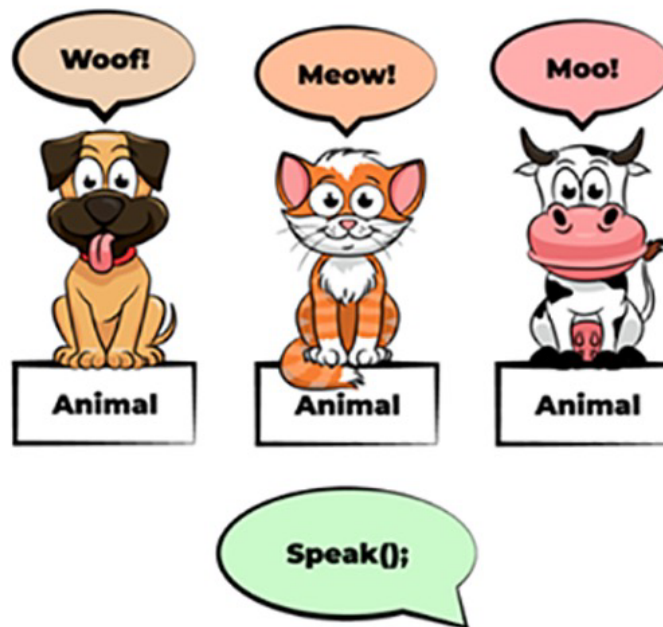
AGENDA

37

- A wee recap of assignment(s) from last time?
- SOLID principles behind OO design
- Abstract classes
- Interfaces
- Polymorphism
- The important stuff
- Assignments

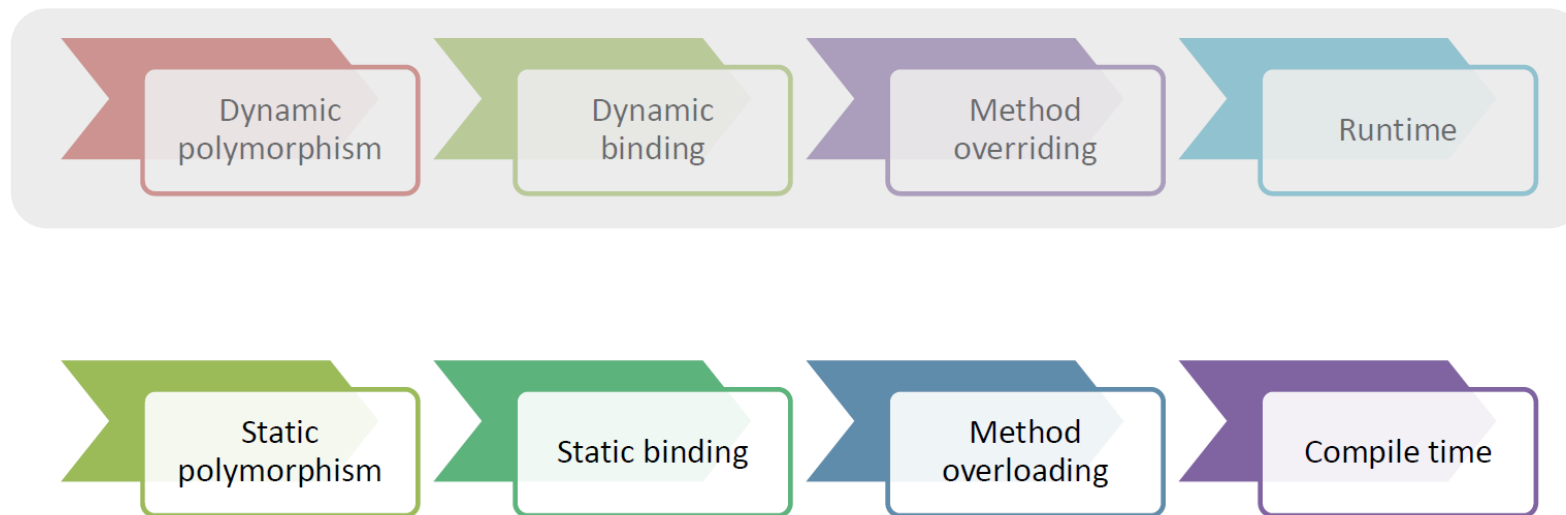
POLYMORPHISM

38



POLYMORPHISM

39

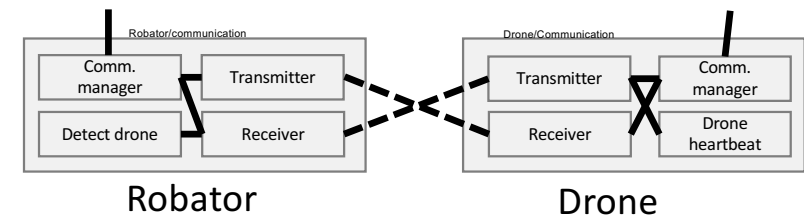


STATIC POLYMORPHISM

40

- Also known as **method overloading**
- Say we write code for the communication manager and just want to have one Transmitter class
 - Robator → drone: we need to send data as well as an ID to ensure we transmit to the correct drone
 - Drone → Robator: since we have only one Robator an ID is a bit superfluous

Note: of course, we could just use ID=0 on the drone side, but not using an ID hints to us, when reading the code, that we do not need an ID – hence on a drone



```
public final class Transmitter {  
  
    // Drone side → Robator (single destination)  
    public void send(byte[] payload) {  
        // send to default/only Robator  
        sendInternal(Destination.ROBOTAR_SINGLETON, payload);  
    }  
  
    // Robator side → Drone (select destination by ID)  
    public void send(String droneId, byte[] payload) {  
        // validate droneId, routing table lookup, etc.  
        Destination dest = resolveDrone(droneId);  
        sendInternal(dest, payload);  
    }  
}
```

STATIC BINDING

- Method overloading = compile-time polymorphism
- In languages like Java, C#, C++, Swift, overloading is resolved by the compiler and not at runtime
- That means the compiler decides which method to call based only on the types of the arguments at the call site
- This is called static binding – and the binding between object reference and class types are done statically when the type of class is known and explicitly declared in advance

```
Bicycle bicycle = new Bicycle()  
Vehicle bicycle2 = new Bicycle()  
Bicycle bicycle3 = new RacerBike()
```

- At compilation time the relation is established, and the how and what of the different objects are defined

METHOD OVERLOADING

- When calling methods from different classes it is sometimes convenient to rely on the same method name but in different ways

```
Bicycle bicycle = new Bicycle()  
bicycle.brake()  
bicycle.brake(frontWheel)  
bicycle.brake(frontWheel, 0.5)
```

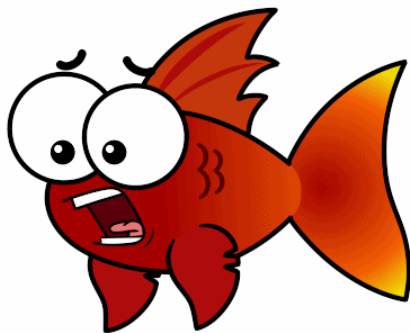
Vs.

```
Bicycle bicycle = new Bicycle()  
bicycle.brake()  
bicycle.brakeFrontWheel()  
bicycle.brakeFrontWheelStr(0.5)
```

- We overload the method and save a lot of complexity
 - We have one method *brake()* instead of three different methods
 - Balance between overloaded and specific methods is a design choice

BREAK – PAUSE(FISK)

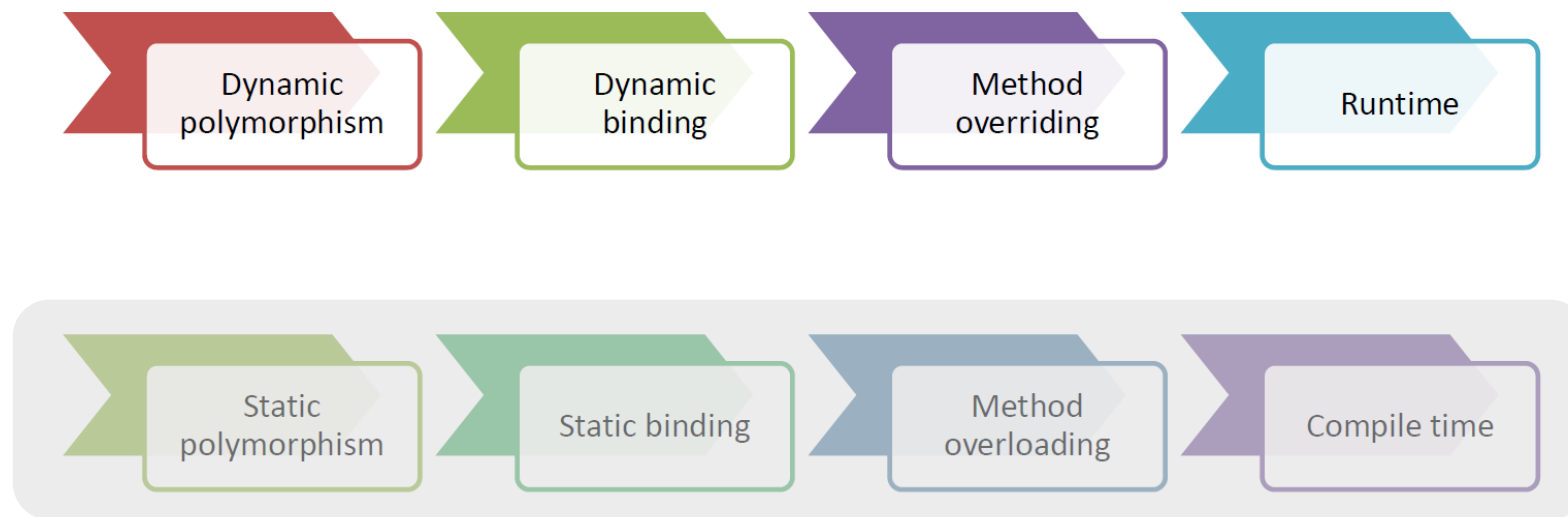
43



Jan H. Mikkelsen, OOAD15, CoCT, 2026

POLYMORPHISM

44

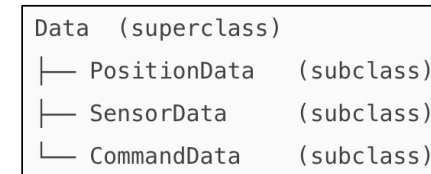


DYNAMIC POLYMORPHISM

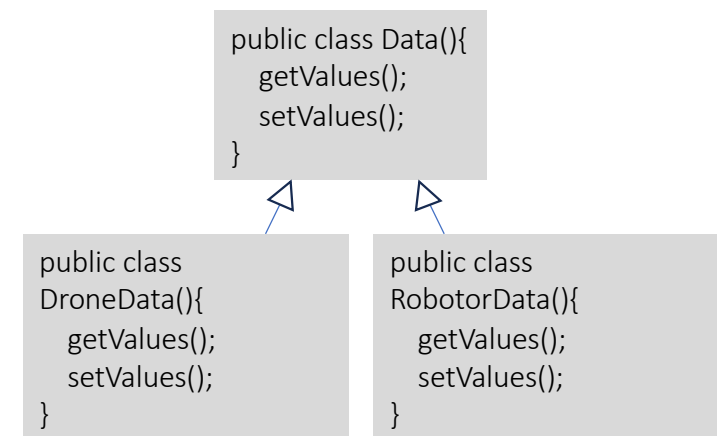
45

- Consider creating a class, Data, to model the data we exchange between the Robator and the drone
- Data could be position, sensor, command etc.
- Sometimes we need to refer to Data in different ways
 - As superclass being a super class
 - As superclass being a subclass
 - As subclass being a subclass
- The appropriate implementation is used - also known as late binding

```
Data d = new PositionData();  
d.send(); // calls PositionData.send() ← late binding
```



Super class = parent class



DYNAMIC BINDING

46

- The method that runs is chosen at runtime based on the *object's actual type*, not on the variable's declared type.
 - Compile-time type decides what is *allowed to be called*
 - Runtime type decides *which implementation* is executed
- The binding between object reference and class types are done dynamically when the type of class is not known and only implicitly declared in advance

```
Vehicle v = new Bicycle();  
v.move(); // Which move() runs?
```

At compile time:

- The compiler only sees Vehicle v
- It checks: "Does Vehicle have a method move()?"
- If yes, the call is allowed

At runtime:

- The object is a Bicycle
- So, Bicycle.move() is the one that executes
- even though the variable type is Vehicle

DYNAMIC BINDING

47

- In this way we can work with different instances and subclasses in the same way, here as a vehicle
- We can, if we want, bind the bicycles differently as needed

```
Vehicle bicycle;  
bicycle = new Bicycle();  
((Bicycle) bicycle).thisIsABicycleInterface();
```

- The compiler only allows calling members that are **declared on the static type** (Vehicle)
- Because `thisIsABicycleInterface()` is **not on Vehicle**, the compiler will reject `bicycle.thisIsABicycleInterface()` unless you **narrow** the type to `Bicycle` (cast)
- `((Bicycle) bicycle)` is an **explicit downcast** to `Bicycle` to access a method that is **not** in `Vehicle`'s API (`thisIsABicycleInterface()`)

METHOD OVERRIDING

- Overriding methods is useful when we need different implementation for the same functionality – often used in class hierarchies

```
Vehicle car = new Car()
car.drive()
Bicycle bicycle = new Bicycle()
bicycle.drive()
Bus bus = new Bus()
bus.drive()
```

vs.

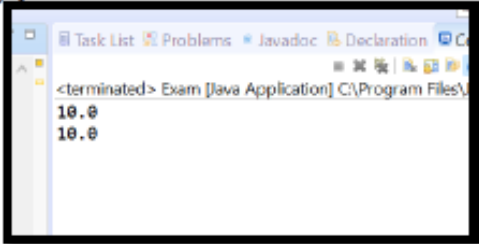
```
Vehicle car = new Car()
car.driveVehicle()
Bicycle bicycle = new Bicycle()
bicycle.rideBicycle()
Bus bus = new Bus()
bus.driveBus()
```

- We obtain an intuition for what we can do, as long we know an object is a Vehicle – then we can **drive()** it
- We do not have this intuition if we do not override base functionality but have to add specific methods to the specific class

OVERRIDING VERSUS OVERLOADING

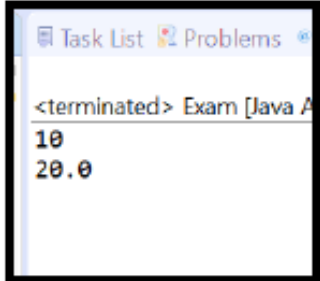
49

```
public class Test {  
    public static void main(String[] args) {  
        A a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    // This method overrides the method in B  
    public void p(double i) {  
        System.out.println(i);  
    }  
}
```



Same type

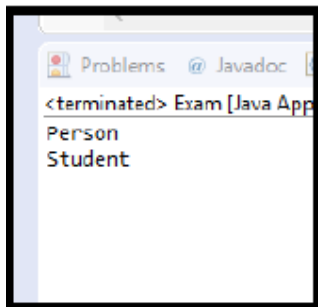
```
public class Test {  
    public static void main(String[] args) {  
        A a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    // This method overloads the method in B  
    public void p(int i) {  
        System.out.println(i);  
    }  
}
```



Diff. types

OVERRIDING VERSUS OVERLOADING

50



```
public class Test {
    public static void main(String[] args) {
        new Person().printPerson();
        new Student().printPerson();
    }
}

class Student extends Person {
    @Override
    public String getInfo() {
        return "Student";
    }
}

class Person {
    public String getInfo() {
        return "Person";
    }

    public void printPerson() {
        System.out.println(getInfo());
    }
}
```

(a)

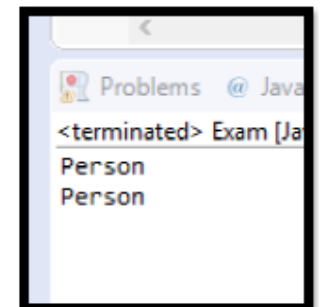
```
public class Test {
    public static void main(String[] args) {
        new Person().printPerson();
        new Student().printPerson();
    }
}

class Student extends Person {
    private String getInfo() {
        return "Student";
    }
}

class Person {
    private String getInfo() {
        return "Person";
    }

    public void printPerson() {
        System.out.println(getInfo());
    }
}
```

(b)



- PrintPerson is defined inside Person, and cannot see the private GetInfo implementation in Student. Thus, we get the implementation in Person each time.

POLYMORPHISM – THE CONSTRUCTOR

51

- For any class, we are required to write a constructor – always!
 - But I don't see one? Why?
- In Java, any class is also an Object (which is a class as well)
- An Object has a default constructor, which is automatically called if we do not write our own
 - In fact, it is always called since it does some needed initialization of the object instance, which is required by java
- But if we wish to call the parent class' constructor because some needed code is in there
 - We can use the keyword ***super()*** to call our parent class constructor, even with parameters
- In this way **WE DON'T NEED TO COPY/PASTE CODE** to all the subclasses of Data
- Use of ***super()*** is not restricted to the constructor...

```
public class Object {  
    defaultConstructor();  
}
```

```
public class Data {  
    protected Byte[] data;  
    public Data(Byte[] d){  
        this.data=d;  
    }  
}
```

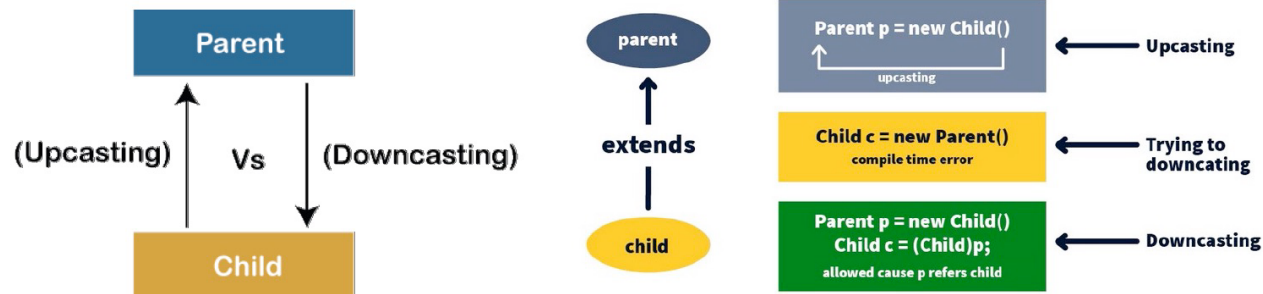
```
public class DroneData {  
    private id=-1;  
    public DroneData(Byte[] d, int id){  
        super(d);  
        this.id = id;  
    }  
}
```

CASTING

52

- Casting allows us to be flexible in our use of our classes
- If we do not cast we cannot use the cast'ed methods and attributes (or we can limit the use of methods and attributes)
- Example:

```
Fruit fruit = new Apple()  
Apple apple1 = (Apple) someUnknownFruit
```



AGENDA

53

- A wee recap of assignment(s) from last time?
- SOLID principles behind OO design
- Abstract classes
- Interfaces
- Polymorphism
- The important stuff
- Assignments

THE IMPORTANT STUFF

- **Abstraction** simplifies complex systems by hiding implementation details
- **Interfaces** define *what* can be done, not *how*; they enable interchangeable components
- **Polymorphism** allows different types to be used through one interface, with behaviour chosen at runtime
- **Overriding** (dynamic binding) enables polymorphism; **overloading** (static binding) does not
- Combining abstraction, interfaces, and polymorphism leads to cleaner, more flexible, and more maintainable software

AGENDA

55

- A wee recap of assignment(s) from last time?
- SOLID principles behind OO design
- Abstract classes
- Interfaces
- Polymorphism
- The important stuff
- **Assignments**



ASSIGNMENTS

56

- 1) Trains can be characterized in many ways and include locomotives and wagons of all sorts (goods, passengers, luxury wagons, armored wagons, dangerous goods etc.). Sketch first a class diagram for defining Trains
- 2) One possible superclass to a Train class could be Vehicle. Define an interface that properly describes what vehicles can do. Let the Train implement the Vehicle interface
- 3) Consider that some of your trains may be electric powered, and some may be diesel (and others). Define an electric Train interface that captures the electric type, e.g. it requires charging, one needs to check status of the battery etc. and implement on a few selected Train classes (or subclasses) that interface
- 4) The electric Train may need to be initiated differently as the normal version of the Train, e.g. starting charge state of the battery which is not present in the diesel version of the train. Use polymorphism to ensure your Train class can support the electric Train version in a generic fashion
- 5) Will there be a difference if the top-level Vehicle class is created as an abstract class? Is this a reasonable choice? Why/why not?
- 6) Create other vehicle types – just a few examples, e.g. bicycles from previous assignments. Which of the operations from previous assignments can you easily perform?